IBM QISKIT FALL FEST 2026

# QUANTUM SPORTS SCHEDULER

Can quantum computers help plan sports tournaments?

---

REGULAR PYTHON | QUANTUM TESTS | REAL IBM COMPUTER | OFFICIAL TOURNAMENT TEST

**Presented by the Quantum Sports Scheduler team**

Our finding: regular Python was faster and more reliable in our tests. We did not prove a quantum speed advantage.

# Why this project matters to me

My sports medicine career inspired this idea

> **I worked in sports medicine and helped at many large wrestling tournaments. I saw how difficult it can be to organize athletes, matches, and several mats at the same time.**

When I saw the tournament scheduling challenge, I wanted to test whether a new kind of computing could help with a problem I knew from experience.

MY EXPERIENCE

## Sports medicine

## Large wrestling tournaments

This inspired our research. We did not organize a real tournament using this software.

**Why I chose this topic: Griselda Bassette**

# Four parts of our project

Each part answers a different question.

## Small league schedule

Four teams: compared regular Python with quantum simulation

## Wrestling tournament

118 matches: made a schedule, then tested a smaller quantum choice

## Real IBM quantum computer

Ran our wrestling QAOA on ibm_fez (after a 2-qubit test)

## Official tournament example

Checked an existing 8-team schedule against the official rules

Our approach: check the math first, then test solutions and check them again.

**Important: CHECKING an official schedule is not the same as CREATING it.**

# Build it, check it, compare it

We checked the rules and results instead of trusting a computer just because it gave an answer.

1 **Choose a scheduling task** → 2 **Turn rules into math** → 3 **Check every possible answer** → 4 **Try Python and quantum** → 5 **Check the final answer**

**262,144**

**Possible answers checked for the 4-team test**

**16,384**

**Possible answers checked for the wrestling test**

QAOA is a team effort between two kinds of computing: a quantum program tries possible answers, and a regular computer helps it improve.

**“A good answer must follow the rules, no matter which computer found it.”**

# Wrestling: 118 matches in 20 rounds

Practice tournament: 12 schools, 14 weight classes, 132 wrestlers

**118**
**Matches**

**6**
**Mats available**

**20**
**Rounds needed**

**98.3%**
**Mat spaces used**

## How Python built the schedule

It respected match order, rest between matches, and rules preventing athletes from being scheduled twice at once. Our checker found no broken rules.

## 20 rounds was the minimum

With 118 matches and 6 mats, at least 20 rounds are needed. Our schedule met that minimum.

This was a made-up example with simpler rules; it did not include consolation matches.

# QUBO + QAOA: pick 6 matches at slot 17

At slot 17, 14 matches are ready. Pick 6: one per mat, one per weight class, most bracket progress.

1 **Start with every answer** → 2 **Mark the good answers** → 3 **Mix the answers** → 4 **Measure many times** → 5 **Python tunes and repeats**

## 14 qubits

**One yes/no choice for each ready match**

## 1 of 16,384

**Only one best answer, confirmed before any quantum run**

QUBO: each match is a yes/no choice. Breaking a rule costs 13 points; bracket progress earns 1 or 2.

**Simulator only: Qiskit Aer • p = 1 and 2 • 5 seeds each • 4,000 final shots**

# Regular Python vs. quantum testing

We compared two small tasks. Quantum tests ran on a simulator, not a real quantum computer.

| COMPARISON | 4-TEAM LEAGUE | WRESTLING CHOICE |
|---|---|---|
| **Quantum test size** | 18 qubits; depths p=1, 2, 3 | 14 qubits; depths p=1, 2 |
| **Successful quantum runs** | Valid: 1 of 5 (p=1), 3 of 5 (p=2), 4 of 5 (p=3) | Best choice: 3 of 5 (p=1), 2 of 5 (p=2) |
| **Valid answers vs. guessing** | Mean 0.08–0.30% vs. 0.002% random | 15% (p=1), 19% (p=2) vs. 8% random |
| **Regular Python** | About 0.00001 sec; valid every time | About 0.00001 sec; best answer every time |
| **Quantum simulator time** | About 2–7 seconds per try | About 1.4–2.8 seconds per try |
| **Another Python method** | About 0.013 sec; valid every time | Full search confirmed the best answer |

## Bottom line: regular Python was faster and more dependable in these tests.

The quantum times are from a simulator running on a normal computer. We only ran a small number of trials.

# Our scheduling QAOA on real IBM hardware

One job on ibm_fez: 14 qubits, 4,000 shots, saved simulator angles (no new optimization)

**13.2%**

**Valid schedules on ibm_fez**

MEASURED: 528 of 4,000 shots

**23.5%**

**Same circuit on the simulator**

Noise model had predicted 17.4%

**7.9%**

**Random guessing**

Hardware was 1.7× higher ($p < 0.001$)

## What this proved

Our algorithm ran on a real quantum chip and beat random guessing. All 423 valid answers passed our checker.

## Noise made it harder

The chip needed 280 two-qubit gates. Noise cut valid answers from 23.5% to 13.2%, and the best answer (score 12) did not appear. A 2-qubit Bell test (95/100 matching) checked our setup first.

Only 13.2% is a hardware measurement; 23.5% (simulator) and 17.4% (noise model) are not. Job ID: dbppooc4148s739v0lsg

# Why try quantum if Python was faster?

We wanted to learn whether a different way of computing could help with hard scheduling choices.

### WHY IT MAY HELP

Tournament rules can be turned into math. A quantum program can adjust the chances of finding a good answer.

### WHY PYTHON WON HERE

Our examples were small. Regular Python solved them almost instantly. Real quantum computers also make errors.

## WHAT WOULD HELP IN THE FUTURE

### A more practical way to model big events

One direct model of the official 8-team example needs 784 yes/no variables, before adding any extras.

### More reliable quantum computers

On the real chip our circuit had 356 layers and 280 two-qubit gates; noise cut valid answers from 23.5% to 13.2%.

### Harder problems and fair tests

Test bigger challenges that regular computers struggle with, using the same rules for both methods.

More qubits do not automatically mean faster or better answers. We would need to test that.

# We checked an official tournament example

We checked a published schedule; we did not create that schedule.

**8**
Teams

**56**
Games in 14 rounds

**161/161**
Rules checked

**0**
Broken rules

## What kinds of rules?

**Home/away capacity (85) • Game timing (31) • Home/away streaks (45)**

## 228 tests passed

46 new tests deliberately break rules to make sure our checker spots mistakes. Unknown rules are also flagged.

The published example is an artificial competition test. Source credit and license information are in data/robinx/SOURCE.md.

**We CHECKED the published schedule. Neither Python nor QAOA created it.**

# What did our team learn?

A hands-on learning project, with results we can check

**WHAT WE BUILT**

A working Python wrestling schedule, quantum experiments, our scheduling QAOA on real IBM hardware, and a checker for an official tournament example.

**WHAT WE FOUND**

Regular Python was faster and more reliable. Quantum beat random guessing on the simulator and on real hardware (13.2% vs. 7.9%), but not Python.

**WHAT WE COULD TRY NEXT**

Test more difficult tournaments, repeat the hardware run, and try error mitigation to reduce the effect of noise.

---

**github.com/bassette143/quantum-sports-scheduler**

**228 passing tests • Five-part Streamlit dashboard**